福州大学

《Linux 操作系统设计实践》实验报告

学号：102401339　姓名：任宝硕　专业：计算机科学与技术　班级：3 班

实验三　网络编程及文件操作

实验环境

本实验在 macOS 环境下完成，使用 C 语言编写，通过 VSCode 终端编译运行。程序使用 TCP 套接口通信，分别通过文件描述符和数据流两种方式读写文件。

实验内容

编写了一个客户端与服务器端通信程序，由 server.c、client.c 和 Makefile 三个文件组成。server.c 负责监听连接、接收消息和发送回复；client.c 负责连接服务器、发送消息和接收回复。双方将收到的内容保存到文件，再从文件读回。Makefile 将两个源文件编译为 server 和 client，并提供 clean 目标清理可执行文件和接收文件。

服务器启动后调用 socket()创建 IPv4 TCP 套接口，设置 SO_REUSEADDR，再调用 bind()绑定本机的 8888 端口。地址使用 INADDR_ANY，表示本机任意 IPv4 地址。随后调用 listen()监听，通过 accept()等待客户端连接。accept()返回一个用于本次通信的文件描述符，原来的描述符仍用于监听。

客户端调用 socket()创建套接口，通过 connect()连接 127.0.0.1:8888。连接失败时，程序通过 perror()输出错误信息，提示先运行./server，并以退出码 1 结束。连接成功后，客户端循环调用 write()发送“Hi, this is client.”，再调用 shutdown(fd,SHUT_WR)关闭发送方向，等待服务器回复。

服务器循环调用 read()接收消息，读到 EOF 后结束接收。程序使用长度为 256 的字符数组，最多保存 255 字节消息，末尾保留空字符。接收出错、消息为空或超过长度限制时，程序输出错误信息并退出。

服务器收到消息后，调用 open()打开 server_recv.txt，使用 O_WRONLY|O_CREAT|O_TRUNC，以只写方式打开文件，不存在时创建，存在时清空。随后调用 write()保存消息，close()关闭文件。服务器再以 O_RDONLY 打开该文件，通过 read()读回内容，用 strcmp() 与收到的消息比较。不一致时输出“文件内容验证失败”，以退出码 1 结束。

文件验证通过后，服务器发送“Hello from server!”，关闭发送方向，再关闭连接和监听套接口。客户端收到回复后，调用 fopen()以"w"模式打开 client_recv.txt，通过 fprintf()写入回复，fclose()关闭文件。随后以"r"模式打开文件，使用 fgets()读回内容，比较后输出结果，关闭套接口并退出。

在 exp3 目录下执行 make，打开两个终端，先运行./server，待其输出等待提示后再运行./client。服务器输出“[服务器] 收到客户端消息：Hi, this is client.”，客户端输出“[客户端] 收到服务器回复：Hello from server!”。双方还会输出从文件读回的内容。运行结束后，server_recv.txt 中为“Hi, this is client.”，client_recv.txt 中为“Hello from server!”。

程序代码

server.c

```c
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8888

int main(void) {
  int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
  if (listen_fd < 0) {
    perror("socket");
    return 1;
  }

  int on = 1; /* 允许重启后重新绑定端口 */
  if (setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &on, sizeof(on)) < 0) {
    perror("setsockopt");
    return 1;
  }

  struct sockaddr_in addr;
  memset(&addr, 0, sizeof(addr));
```

server.c

```
  addr.sin_family = AF_INET;
  addr.sin_addr.s_addr = htonl(INADDR_ANY); /* 本机任意地址 */
  addr.sin_port = htons(PORT);
  if (bind(listen_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
    perror("bind");
    return 1;
  }
  if (listen(listen_fd, 1) < 0) {
    perror("listen");
    return 1;
  }
  printf("[服务器] 监听端口 %d，等待客户端...\n", PORT);

  fflush(stdout);

  int conn_fd = accept(listen_fd, NULL, NULL);
  if (conn_fd < 0) {
    perror("accept");
    return 1;
  }

  /* 单次短文本验证，最多 255 字节；大文件传输时改为分块处理。 */
  char buf[256] = {0};
  size_t used = 0;
  ssize_t n;
  while ((n = read(conn_fd, buf + used, sizeof(buf) - 1 - used)) > 0) {
    used += n;
    if (used == sizeof(buf) - 1) {
      char extra;
      n = read(conn_fd, &extra, 1);
      if (n > 0) {
        fprintf(stderr, "消息超过 255 字节\n");
        return 1;
      }
      break;
    }
  }
  if (n < 0 || used == 0) {
    fprintf(stderr, "接收失败或消息为空\n");
    return 1;
  }
  printf("[服务器] 收到客户端消息：%s\n", buf);

  int file_fd = open("server_recv.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
  if (file_fd < 0) {
    perror("open");
    return 1;
  }
  if (write(file_fd, buf, used) != (ssize_t)used) {
    perror("write");
    return 1;
  }
```

server.c

```
  if (close(file_fd) < 0) {
    perror("close file");
    return 1;
  }
  printf("[服务器] 消息已保存到 server_recv.txt\n");

  char file_buf[256] = {0};
  file_fd = open("server_recv.txt", O_RDONLY);
  if (file_fd < 0) {
    perror("open");
    return 1;
  }
  if (read(file_fd, file_buf, sizeof(file_buf) - 1) < 0) {
    perror("read");
    return 1;
  }
  if (close(file_fd) < 0) {
    perror("close file");
    return 1;
  }
  if (strcmp(buf, file_buf) != 0) {
    fprintf(stderr, "文件内容验证失败\n");
    return 1;
  }
  printf("[服务器] 从文件读回：%s\n", file_buf);

  const char *reply = "Hello from server!";
  /* TCP 可能只发送部分数据，循环直到整条消息发完。 */
  for (size_t sent = 0; sent < strlen(reply);) {
    ssize_t n = write(conn_fd, reply + sent, strlen(reply) - sent);
    if (n <= 0) {
      perror("write socket");
      return 1;
    }
    sent += n;
  }
  if (shutdown(conn_fd, SHUT_WR) < 0) { /* 用发送方向的 EOF 标记消息结束 */
    perror("shutdown");
    return 1;
  }
  printf("[服务器] 已发送回复：%s\n", reply);

  close(conn_fd);
  close(listen_fd);
  return 0;
}
```

client.c

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>

```

client.c

```c
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8888

int main(void) {
  int fd = socket(AF_INET, SOCK_STREAM, 0);
  if (fd < 0) {
    perror("socket");
    return 1;
  }

  struct sockaddr_in addr;
  memset(&addr, 0, sizeof(addr));
  addr.sin_family = AF_INET;
  addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK); /* 127.0.0.1 */
  addr.sin_port = htons(PORT);
  if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
    perror("connect（请先运行 ./server）");
    return 1;
  }

  const char *msg = "Hi, this is client.";
  /* TCP 可能只发送部分数据，循环直到整条消息发完。 */
  for (size_t sent = 0; sent < strlen(msg);) {
    ssize_t n = write(fd, msg + sent, strlen(msg) - sent);
    if (n <= 0) {
      perror("write socket");
      return 1;
    }
    sent += n;
  }
  if (shutdown(fd, SHUT_WR) < 0) { /* 用发送方向的 EOF 标记消息结束 */
    perror("shutdown");
    return 1;
  }
  printf("[客户端] 已发送消息：%s\n", msg);

  /* 单次短文本验证，最多 255 字节；大文件传输时改为分块处理。 */
  char buf[256] = {0};
  size_t used = 0;
  ssize_t n;
  while ((n = read(fd, buf + used, sizeof(buf) - 1 - used)) > 0) {
    used += n;
    if (used == sizeof(buf) - 1) {
      char extra;
      n = read(fd, &extra, 1);
      if (n > 0) {
        fprintf(stderr, "消息超过 255 字节\n");
        return 1;
      }
      break;
```

client.c

```
    }
  }
  if (n < 0 || used == 0) {
    fprintf(stderr, "接收失败或消息为空\n");
    return 1;
  }
  printf("[客户端] 收到服务器回复：%s\n", buf);

  FILE *fp = fopen("client_recv.txt", "w");
  if (!fp) {
    perror("fopen");
    return 1;
  }
  if (fprintf(fp, "%s", buf) < 0) {
    perror("fprintf");
    return 1;
  }
  if (fclose(fp) == EOF) {
    perror("fclose");
    return 1;
  }
  printf("[客户端] 回复已保存到 client_recv.txt\n");

  fp = fopen("client_recv.txt", "r");
  if (!fp) {
    perror("fopen");
    return 1;
  }
  char check[256] = {0};
  if (!fgets(check, sizeof(check), fp)) {
    fprintf(stderr, "文件读取失败\n");
    return 1;
  }
  if (fclose(fp) == EOF) {
    perror("fclose");
    return 1;
  }
  if (strcmp(buf, check) != 0) {
    fprintf(stderr, "文件内容验证失败\n");
    return 1;
  }
  printf("[客户端] 从文件读回：%s\n", check);

  close(fd);
  return 0;
}
```

Makefile

```
all: server client

server: server.c
    gcc -o server server.c
```

Makefile

```
05
06 client: client.c
07     gcc -o client client.c
08
09 clean:
10     rm -f server client server_recv.txt client_recv.txt
11
12 .PHONY: all clean
```

运行截图

```
baoshuo → ~/projects/202601-linux-exp (master ✗) $ cd /Users/baoshuo/projects/202601-linux-exp/exp3
make -B
./server &
server_pid=$!
sleep 1
./client
wait $server_pid
printf 'Hi, this is client.' | cmp - server_recv.txt && printf 'server_recv.txt: PASS\n'
printf 'Hello from server!' | cmp - client_recv.txt && printf 'client_recv.txt: PASS\n'

gcc -o server server.c
gcc -o client client.c
[1] 56727
[服务器] 监听端口 8888，等待客户端...
[客户端] 已发送消息：Hi, this is client.
[服务器] 收到客户端消息：Hi, this is client.
[服务器] 消息已保存到 server_recv.txt
[服务器] 从文件读回：Hi, this is client.
[服务器] 已发送回复：Hello from server!
[客户端] 收到服务器回复：Hello from server!
[客户端] 回复已保存到 client_recv.txt
[客户端] 从文件读回：Hello from server!
[1]  + 56727 done       ./server
server_recv.txt: PASS
client_recv.txt: PASS
baoshuo → ~/projects/202601-linux-exp/exp3 (master ✗) $
```

截图显示编译输出和消息收发结果。服务器在后台监听，客户端发送消息并接收回复。双方输出文件读回结果，随后使用 cmp 比较两个接收文件与预期消息，结果均为 PASS。

实验总结

1. 实验程序由 server 和 client 两个进程完成一次消息往返。本次运行使用回环地址 127.0.0.1，通信发生在同一台计算机内。两个进程均以退出码 0 结束，接收文件的内容与发送的消息一致。
2. socket()返回套接口的文件描述符，程序通过 read()和 write()收发数据。服务器需要先绑定地址和端口，再监听并接收连接；客户端使用服务器地址和端口发起连接。sockaddr_in 保存 IPv4 地址和端口，htons()和 htonl()将数值转换为网络字节序。

3. TCP 是字节流，一次 write()发出的消息可能需要多次 read()才能读完。程序循环发送和接收，用 shutdown(fd,SHUT_WR)表示本方发送结束。对方读完已发送的数据后，read()返回 0。关闭发送方向后仍可接收数据，所以客户端可以在发送结束后等待回复。
4. 服务器使用文件描述符读写文件，客户端使用 FILE 指针操作数据流。fprintf()的输出可能先保存在缓冲区中，fclose()会刷新缓冲区并关闭文件，因此写入和关闭的返回值都需要检查。两边保存文件时都会清空旧内容，上次运行的消息不会保留。
5. 程序只接收一个客户端，完成一次消息交换后退出，没有设置等待超时。文件读取也按本次短文本处理：服务器调用一次 read()，客户端调用一次 fgets()。